UNIVERSIDAD DE CALDAS

Programación Web II · Parcial de la Unidad 2
Interactividad avanzada, almacenamiento local y validación avanzada de formularios

Estudiantes: Juan Manuel Orozco y Víctor Martínez
Fecha: _____

SECCIÓN I. SELECCIÓN MÚLTIPLE

Preguntas 1 a 20 · Respuesta correcta

1. Respuesta: B. badInput: ocurre cuando el navegador no puede interpretar lo que se escribió como un número.

2. Respuesta: D. Se debe volver a validar el campo de confirmación cuando cambia la contraseña, si ese campo ya fue tocado.

3. Respuesta: A. validity.valid será false porque customError también indica que existe un error de validación.

4. Respuesta: C. El evento invalid no burbujea; por eso se debe escuchar usando la fase de captura.

5. Respuesta: B. novalidate evita que el navegador bloquee el envío automáticamente, pero el código todavía puede consultar validity.

6. Respuesta: A. innerHTML puede permitir una inyección de código HTML (XSS). Es más seguro mostrar el mensaje con textContent.

7. Respuesta: D. El campo sigue inválido hasta limpiar el error con setCustomValidity("").

8. Respuesta: C. patternMismatch aparece antes que tooShort en el orden indicado, por eso se asigna el mensaje sobre los caracteres permitidos.

9. Respuesta: C. El mensaje explica qué debe escribir y muestra un ejemplo claro del formato esperado.

10. Respuesta: A. Validar al salir del campo y corregir mientras se escribe después de un error evita interrumpir el trabajo.

11. Respuesta: D. Los mensajes junto a cada campo, asociados con ARIA y controlados por código, permiten mostrar varios errores claramente.

12. Respuesta: B. :invalid se evalúa continuamente. Es mejor usar :user-invalid o una clase que controle cuándo mostrar el error.

13. Respuesta: C. El mensaje hecho solo con CSS puede no ser anunciado por lectores de pantalla y no cambia según el error específico.

14. Respuesta: A. El placeholder permite distinguir visualmente un campo vacío de uno que ya contiene texto.

15. Respuesta: D. Las clases o atributos controlados por código permiten mostrar tanto errores del servidor como errores entre varios campos.

16. Respuesta: B. La animación debe reducirse o desactivarse cuando la persona tenga activada la preferencia de movimiento reducido.

17. Respuesta: C. aria-describedby relaciona el mensaje con el campo y aria-invalid indica que el valor tiene un error.

18. Respuesta: A. role=status sirve para informar que se está enviando y role=alert para anunciar el resumen de errores.

19. Respuesta: D. WCAG 1.4.1 indica que no se debe usar solamente el color; también se debe incluir texto o un icono.

20. Respuesta: B. La validación del navegador ayuda a la persona, pero no reemplaza la validación del servidor ni el proceso ETL.

SECCIÓN II. RESPUESTA ABIERTA

21. Explique el ciclo de validación nativa del navegador.

Cuando una persona intenta enviar un formulario, el navegador revisa los campos y sus reglas, como required, min, max o pattern. Si encuentra un campo inválido, dispara el evento invalid y consulta sus propiedades en validity para saber qué regla no se cumple. Después evita el envío y puede mostrar un mensaje de error. Si el formulario tiene novalidate, el navegador no hace ese bloqueo automático ni muestra su aviso, pero validity sigue funcionando. El código puede revisar los errores y usar métodos como checkValidity() o reportValidity().

22. Compare :user-invalid con las clases o atributos controlados por código.

La pseudo-clase :user-invalid permite mostrar un error después de que la persona interactúa con el campo. Es sencilla y sirve para validaciones comunes del navegador. En cambio, las clases y atributos como .es-invalido o aria-invalid="true" se pueden cambiar desde JavaScript. Para un formulario complejo elegiría las clases controladas por código, porque permiten mostrar también errores que llegan del servidor y errores que dependen de varios campos.

23. ¿Por qué el color rojo no es suficiente para indicar un error?

Porque algunas personas no distinguen todos los colores y podrían no notar el error. Además del borde rojo, pondría un mensaje claro y un icono. Usaría aria-invalid="true" para marcar el campo y aria-describedby para relacionarlo con su mensaje. También usaría regiones vivas para anunciar avisos importantes y buen contraste para que el texto se lea. Después de enviar el formulario con errores, llevaría el foco al primer campo que necesita corrección o a un resumen de errores.

24. ¿Por qué la validación del navegador no sustituye la del servidor ni el ETL?

La validación del navegador ayuda a evitar errores al escribir, pero una persona puede desactivarla o enviar datos por otros medios. Por eso el servidor debe revisar nuevamente la información antes de guardarla. Más adelante, el proceso ETL extrae, transforma y carga los datos, y puede organizarlos o corregir problemas. Si los datos llegan bien desde el formulario, se necesita menos limpieza. Si el servidor devuelve un error, lo mostraría junto al campo correspondiente, con un mensaje claro y usando textContent para insertarlo de forma segura.

25. ¿En qué momento conviene mostrar los errores de validación?

No mostraría errores apenas se carga la página, porque puede confundir. Validaría al salir de cada campo para revisar lo que la persona escribió. Si ya apareció un error, volvería a revisar mientras escribe para que el mensaje desaparezca cuando lo corrija. Al enviar, revisaría todos los campos y mostraría los errores pendientes. Esta combinación evita interrumpir a la persona y permite corregir antes de terminar el registro.

SECCIÓN III. APLICACIÓN MEDIANTE EJEMPLOS

26. Ejemplo de formulario con mensajes personalizados.

Usaría el formulario de una tienda en línea llamada Compra Fácil:
• **Correo:** tipo email. Si el formato no es correcto se activa typeMismatch. Mensaje: “Escribe un correo válido. Ejemplo: nombre@correo.com”.
• **Edad:** tipo number, mínimo 18. Si escribe una edad menor se activa rangeUnderflow. Mensaje: “Debes tener mínimo 18 años para registrarte”.
• **Contraseña:** mínimo 8 caracteres. Si escribe menos se activa tooShort. Mensaje: “Escribe una contraseña de mínimo 8 caracteres”.
Los mensajes son claros, dicen cómo corregir el error y dan ejemplos cuando hace falta.

27. Sistema de inventario para una empresa.

En un inventario donde se registran muchos productos usaría estos campos:
• **Código:** input text con pattern="[A-Z]{3}-[0-9]{4}". Ejemplo válido: REP-1234. Mensaje: “Escribe tres letras, un guion y cuatro números”.
• **Cantidad:** input number con min="0", max="5000" y step="1". Mensaje: “Escribe una cantidad entre 0 y 5000, sin decimales”.
• **Precio:** input number con min="0" y step="0.01". Mensaje: “Escribe un precio mayor o igual a cero”.
Validaría al salir de cada campo, volvería a validar mientras se corrige un error y revisaría todo al enviar. Así el personal puede trabajar sin interrupciones constantes.

28. Ejemplo de retroalimentación visual con CSS.

Definiría variables como --borde, --error, --correcto y --fondo. El estado neutro tendría un borde gris; al enfocar el campo tendría un borde destacado; cuando sea válido mostraría un borde verde y cuando sea inválido un borde rojo acompañado de texto o un icono. Usaría :focus-visible para el enfoque y :user-valid y :user-invalid para los estados cuando el navegador los soporte. Como respaldo, JavaScript puede agregar clases como .valido y .es-invalido. Para respetar a quienes prefieren menos movimiento, usaría @media (prefers-reduced-motion: reduce) para quitar o reducir las animaciones.

29. Caso en el que el servidor rechaza un dato.

En un registro, una persona puede escribir un correo con formato correcto que ya está registrado. El servidor revisa la base de datos y responde con un error asociado al campo, por ejemplo: campo: “correo”; mensaje: “Este correo ya está registrado”. La página mostraría el mensaje debajo del correo, pondría aria-invalid="true" y lo relacionaría con aria-describedby. También llevaría el foco al campo para que la persona lo corrija. Insertaría el mensaje con textContent y no con innerHTML, para evitar inyección de código. En un inicio de sesión usaría un mensaje general, sin confirmar si un correo o una cuenta existe.

30. Captura de datos que alimenta un proceso ETL.

Una empresa registra los datos de sus empleados para después generar informes. En el navegador revisaría que los campos obligatorios estén completos, que el correo tenga formato correcto y que las fechas y números tengan valores permitidos. En el servidor comprobaría nuevamente los datos, revisaría que el documento no esté repetido y que la persona tenga permiso para hacer el registro. Después, el proceso ETL extraería los datos, los transformaría para organizarlos y los cargaría en el sistema de informes. Validar desde el inicio reduce errores y trabajo de limpieza, pero no reemplaza las tareas propias del ETL.